

PROJECT REPORT

Snake Water Gun Game

A GUI-based Python application using Tkinter

Submitted by	SAMEERA SINGH
Registration No.	26BCE10025
Course / Section	CSE CORE
Faculty / Mentor	Dr. SIVABALAN KR.
Academic Year	2026-27

Project type: Desktop GUI application

Technology: Python 3 + Tkinter

1. Introduction

Snake Water Gun is a simple decision-based game in which a player selects one of three choices: Snake, Water, or Gun. The computer makes its own choice, and the two choices are compared using the standard rules of the game:

- Snake drinks Water (Snake wins)
- Water douses Gun (Water wins)
- Gun shoots Snake (Gun wins)
- Identical choices result in a Draw

The purpose of this project is to implement the game as a small desktop application instead of a command-line program. The application uses Python and the Tkinter library to provide buttons, labels, a result area, and a running score. The project also demonstrates basic modular programming by dividing the program into separate Python files.

The final application is intentionally simple. It focuses on clear interaction, readable code, and the use of functions, modules, random selection, conditional statements, and Tkinter callbacks.

Project objectives

- Build a playable Snake Water Gun game in Python.
- Create a graphical user interface using Tkinter.
- Generate the computer's move randomly.
- Compare the player's move with the computer's move.
- Display the winner of every round.
- Maintain and display the score.
- Separate the program into logical modules for easier maintenance.

2. Problem Statement

A basic command-line Snake Water Gun program can perform the game logic, but it is less interactive for a beginner-level desktop project. The problem is to design and implement a simple graphical application where a user can select a move through buttons and immediately see the computer's move, the round result, and the updated score.

The application should be easy to understand and operate without requiring the user to type commands. It should also keep the game logic separate from the user interface so that individual parts can be changed without rewriting the whole program.

Scope

- Single-player game against a computer-controlled opponent.
- Three moves: Snake, Water, and Gun.
- One round is played whenever the user clicks a move button.
- The score remains visible while the application is running.
- No login, database, internet connection, or external game server is required.

3. Functional Requirements

The functional requirements describe what the application should do during normal use.

ID	Requirement	Description
FR1	Start game	Open a Tkinter window containing the game controls.
FR2	Select move	Allow the player to choose Snake, Water, or Gun.
FR3	Computer move	Generate a random computer choice from the three valid moves.
FR4	Compare moves	Apply the standard rules to determine the round result.
FR5	Show result	Display both choices and the winner/draw message.
FR6	Update score	Increase the appropriate score after a win.
FR7	Continue game	Allow the player to play additional rounds without restarting.

4. Non-functional Requirements

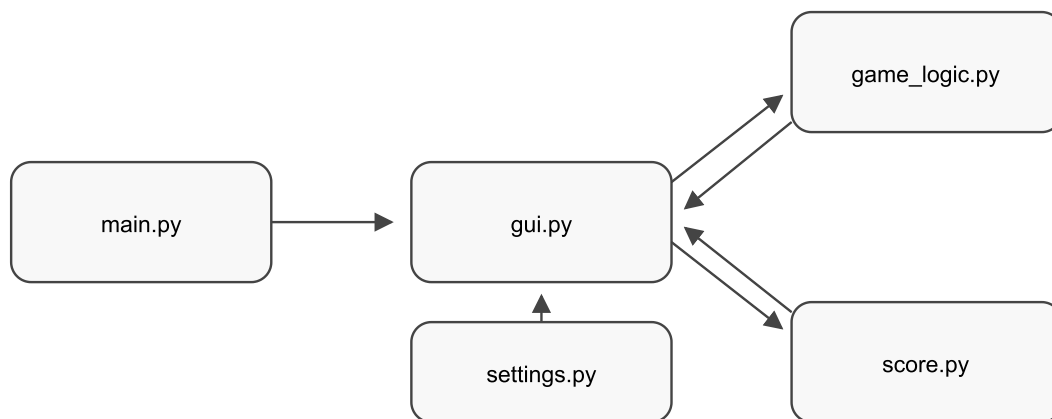
- **Usability:** Buttons and result labels should be clear enough for a first-time user.
- **Performance:** A round should be processed immediately after a button click.
- **Reliability:** The program should accept only the three predefined choices through the GUI.
- **Maintainability:** Game logic, score handling, settings, and GUI code are kept in separate modules.
- **Portability:** The project should run on systems where Python 3 and Tkinter are available.
- **Simplicity:** The interface should contain only the controls needed to play the game.

These requirements are suitable for a small educational desktop application. The project does not require a network connection or persistent storage.

5. System Architecture

The project follows a simple modular architecture. The user interacts with the Tkinter interface in `gui.py`. When a button is pressed, `gui.py` calls the functions in `game_logic.py`. The score module records the result, while `settings.py` contains common constants such as the list of valid choices.

Class / Component Diagram



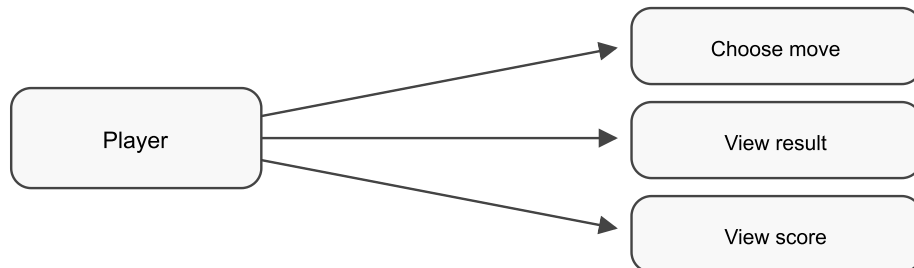
Module responsibilities

- **main.py**: starts the Tkinter event loop.
- **gui.py**: creates the window, buttons, labels, and handles button events.
- **game_logic.py**: selects the computer move and determines the round result.
- **score.py**: stores and updates the player and computer scores.
- **settings.py**: stores shared configuration values.

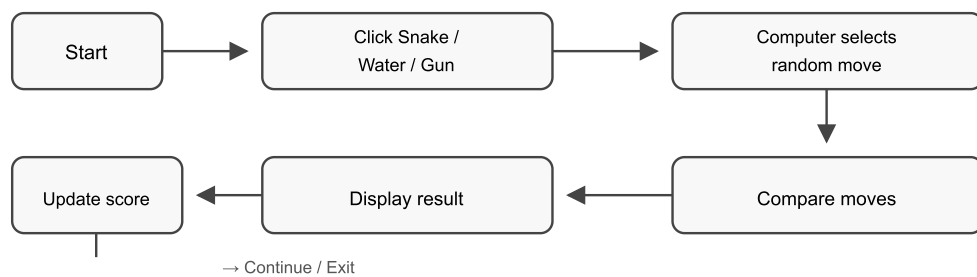
6. Design Diagrams

6.1 Use Case Diagram

The player is the only external actor. The main use cases are selecting a move, viewing the result, and viewing the score.

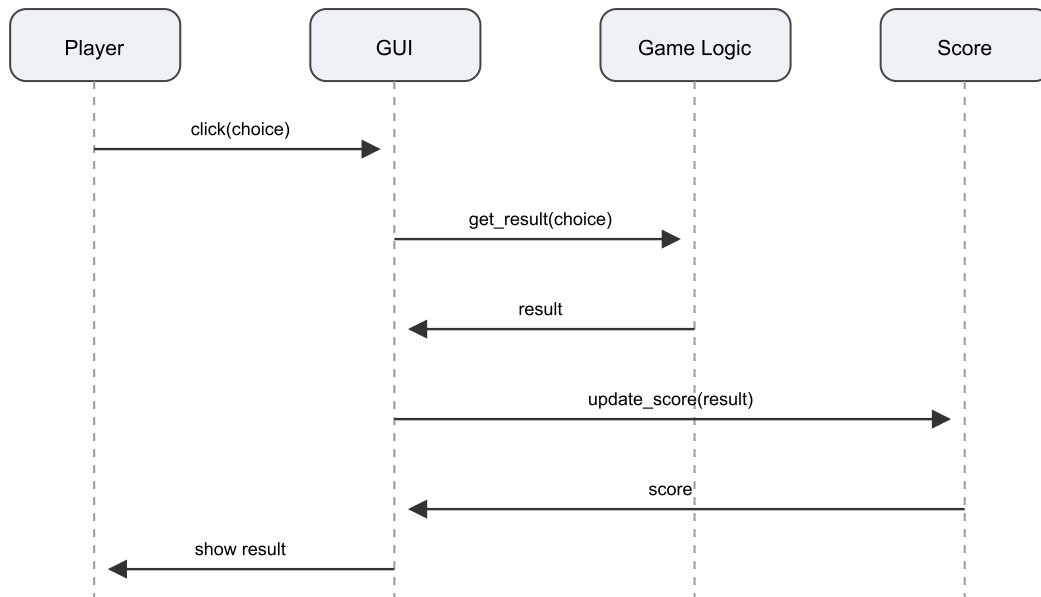


6.2 Workflow Diagram



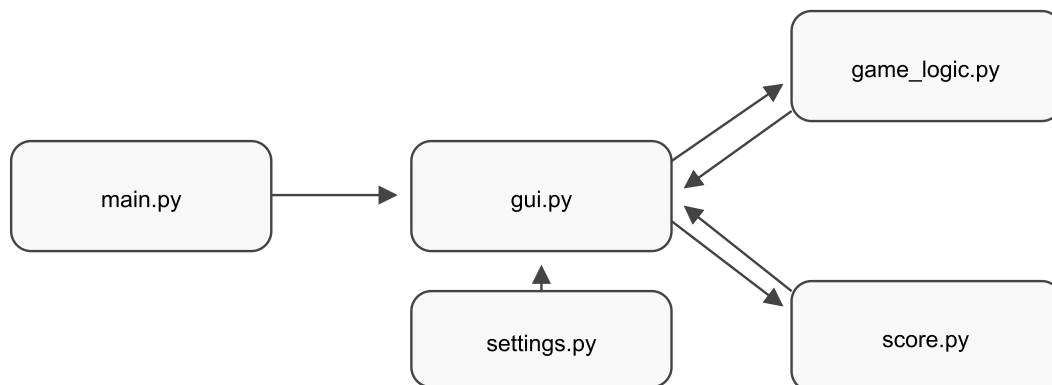
6.3 Sequence Diagram

The sequence below shows the main interaction after the player clicks one of the three buttons.



6.4 Class / Component Diagram

The application is primarily function-based rather than class-based, so the diagram represents the Python modules as components.



6.5 ER Diagram

An ER diagram is not required for this project because the application does not use a database or persistent storage. Scores exist only while the program is running.

7. Design Decisions & Rationale

Choice of Tkinter: Tkinter is included with standard Python installations in many environments and is sufficient for a small desktop GUI. It provides buttons, labels, frames, and event handling without requiring a large framework.

Use of modules: The program is split into five files so that the GUI code does not contain all the game rules. This makes the project easier to read and makes individual parts easier to test.

Random computer move: The random module is used to select one valid move. This prevents the computer from always making the same choice.

Callback with lambda: Each button uses a small lambda expression so the selected string can be passed to `play()`. For example, `command=lambda: play("Snake")` waits until the button is clicked before calling the function.

No database: The game is short-lived and does not need user accounts or saved data. Keeping the score in memory keeps the design simple.

8. Implementation Details

The implementation uses Python functions and Tkinter widgets. The central flow starts when the user clicks a button. The `play()` function receives the selected move, obtains a random computer move, determines the result, updates the score, and changes the labels in the GUI.

Project files

File	Purpose
<code>main.py</code>	Starts the Tkinter main loop.
<code>gui.py</code>	Creates the interface and handles user interaction.
<code>game_logic.py</code>	Contains random selection and game rules.
<code>score.py</code>	Maintains the running scores.
<code>settings.py</code>	Stores common constants.

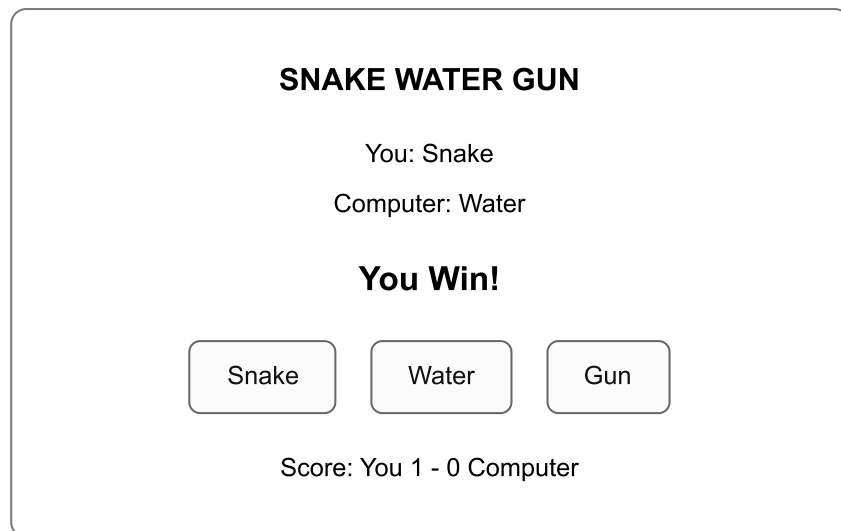
Important implementation logic

```
def get_computer_choice(): return random.choice(CHOICES) def get_result(player, computer): if player == computer: return "It's a Draw!" if (player == "Snake" and computer == "Water") or \ (player == "Water" and computer == "Gun") or \ (player == "Gun" and computer == "Snake"): return "You Win!" return "Computer Wins!"
```

The GUI then calls these functions and updates the labels. This keeps the comparison rules in one place instead of repeating them inside every button callback.

9. Screenshots / Results

The following is a representation of the expected application screen. After running the project, an actual screenshot of the working application can be inserted here for the final submission.



Example result: if the player selects Snake and the computer selects Water, the application displays "You Win!" and increments the player's score.

Suggested evidence for final submission

- Initial game window before a move is selected.
- A round showing the player's and computer's choices.
- A result showing a win, loss, or draw.
- The score after several rounds.

10. Testing Approach

Testing was planned around the three possible outcomes: win, loss, and draw. The interface was also checked to make sure each button triggers the correct choice.

Test	Input / Situation	Expected Result
T1	Snake vs Water	Player wins and player score increases.
T2	Water vs Gun	Player wins and player score increases.
T3	Gun vs Snake	Player wins and player score increases.
T4	Water vs Snake	Computer wins and computer score increases.
T5	Gun vs Water	Computer wins and computer score increases.
T6	Snake vs Gun	Computer wins and computer score increases.
T7	Same choice	Draw message and scores remain unchanged.
T8	Repeated clicks	Each click starts another round and updates the display.
T9	Application start	Window opens with initial labels and score.

The tests focus on observable behavior rather than the internal implementation. Since the computer move is random, a test such as Snake vs Water can be performed by repeating rounds until the required combination appears, or by temporarily controlling the random value during development.

11. Challenges Faced

- **Connecting Tkinter buttons to functions:** The button command needs a callable function. Lambda was used when a value such as Snake, Water, or Gun had to be passed to the play function.
- **Separating the modules:** Moving code into multiple files required imports to be arranged correctly and all files to remain in the same project folder.
- **Keeping the score updated:** The score variables need to be changed after each round and the GUI label must then be refreshed.
- **Handling all game combinations:** The comparison logic has to cover all possible combinations without incorrectly declaring a winner.
- **Designing a simple interface:** The interface was kept small so that the player can understand the game without extra controls.

12. Learnings & Key Takeaways

This project helped in understanding how a Python program can be converted into a basic desktop application. Instead of using only print statements and input(), the program responds to GUI events such as button clicks.

- Learned the basic structure of a Tkinter application.
- Understood how functions can be connected to GUI events.
- Practiced if-elif-else decision making.
- Used the random module for computer-generated choices.
- Learned how Python modules can divide a project into smaller files.
- Understood the role of lambda in Tkinter callbacks.
- Practiced testing different outcomes and edge cases.

The main takeaway is that even a small game benefits from separating the user interface, game rules, configuration, and score handling.

13. Future Enhancements

The current version provides the basic game functionality. The following improvements could be added in a later version:

- Add images or icons for Snake, Water, and Gun.
- Add a Reset Score button.
- Add a fixed number of rounds such as best of 3 or best of 5.
- Add sound effects for button clicks and results.
- Add a start screen and a more polished visual theme.
- Save match history to a file.
- Add a two-player mode on the same computer.
- Add a statistics page showing wins, losses, and draws.

These enhancements can be added without changing the basic rules of the game. The modular structure makes it easier to extend the project later.

14. References

1. Python Software Foundation. Python Documentation The Python Standard Library.
<https://docs.python.org/3/>
2. Python Software Foundation. tkinter Python interface to Tcl/Tk.
<https://docs.python.org/3/library/tkinter.html>
3. Python Software Foundation. random Generate pseudo-random numbers.
<https://docs.python.org/3/library/random.html>
4. Python Software Foundation. tkinter.ttk Tk themed widgets.
<https://docs.python.org/3/library/tkinter.ttk.html>
5. Course/project instructions provided for the project report structure.

Appendix: Suggested repository structure

```
SnakeWaterGun/ main.py gui.py game_logic.py score.py settings.py
```

The report is based on the five-file version of the game developed for this project. Student-specific information and actual application screenshots can be added before submission.